Hands-on session with GitHub Copilot & Copilot CLI

FIE463: Numerical Methods in Macroeconomics and Finance using Python

Richard Foltyn

NHH Norwegian School of Economics

March 5, 2026

Contents

1 AI coding assistants
- Ways to interact/code with LLMs
- Pitfalls & legal issues

2 Hands-on session with Copilot
- GitHub Copilot Chat in VS Code
- GitHub Copilot CLI
- Best practices
- Tutorials

Plan for today

Introduction to AI coding assistants:

- What tools are out there and how do we use them effectively?
- Pitfalls

Programming tasks to solve with GitHub Copilot:

- Mandatory assignment
- Exercises 1 & 2 for workshop 7
- Python program to fetch and summarize company info

AI coding assistants

Ways to interact/code with LLMs

1. **Web browser**
   - Inefficient workflow, LLMs cannot create/modify files on your computer
2. **Inside your IDE** (e.g., VS Code, PyCharm, etc)
   - Integrated via extensions: GitHub Copilot Chat, Google Code Assist, Codex
   - *Edit mode* works on individual files
   - *Agent mode* can run almost anything on your computer
3. **On the command line** (CLI)
   - GitHub Copilot CLI, Google Gemini CLI, Claude Code, OpenAI Codex CLI, OpenRouter.ai
   - Usually in *Agent mode*, can run almost anything on your computer
   - Can be scripted (called programmatically) to perform repeated tasks
4. **Via APIs** (Application Programming Interfaces)
   - Useful to run a large number of tasks non-interactively
   - Can only be used programmatically
   - Requires an API key (may not be included in all subscriptions)
   - Useful for research: Query LLMs 50,000 times to document biases
     "The Worth of a «Wo»: Gender Bias in Financial Advice from LLMs" [SSRN link]

Vibe coding = LLM writes the code based on non-technical instructions

Andrej Karpathy
@karpathy

There's a new kind of coding I call "vibe coding", where you fully give in to the vibes, embrace exponentials, and forget that the code even exists. It's possible because the LLMs (e.g. Cursor Composer w Sonnet) are getting too good. Also I just talk to Composer with SuperWhisper so I barely even touch the keyboard. I ask for the dumbest things like "decrease the padding on the sidebar by half" because I'm too lazy to find it. I "Accept All" always, I don't read the diffs anymore. When I get error messages I just copy paste them in with no comment, usually that fixes it. The code grows beyond my usual comprehension, I'd have to really read through it for a while. Sometimes the LLMs can't fix a bug so I just work around it or ask for random changes until it goes away. It's not too bad for throwaway weekend projects, but still quite amusing. I'm building a project or webapp, but it's not really coding - I just see stuff, say stuff, run stuff, and copy paste stuff, and it mostly works.

12:17 AM · Feb 3, 2025 · **5.1M** Views

Pitfalls

1. LLMs are not deterministic (not even for the same prompt)
   - Your suggestions will be different from mine
   - You will get different suggestions each time
   - Suggestions depend on the context (existing code in your workspace)
   - The complexity of the generated code varies (sometimes the solution uses functions, sometimes it doesn't, etc.)
2. The quality of generated code depends on how much public code exists for a given language
   - Better results with widely used languages like Python and JavaScript
   - Poor results with more exotic/niche/obsolete languages
3. Code suggestions might be outright wrong:
   - The proposed algorithm may not solve the problem
   - Suggestions may use functions or APIs that don't actually exist
4. Code suggestions might use functions or APIs that are deprecated
   - Can be a problem with pandas, which frequently changes APIs

$\Longrightarrow$ **Don't use AI for tasks you can't solve yourself**

Legal issues

- LLMs may infringe on copyright: the corpus of training data may impose license restrictions
- VS Code Copilot **does not** run on your computer
  - Everything you enter & code files will be uploaded to servers operated by Microsoft/Google/OpenAI/Anthropic/Amazon/etc.
  - You may not have permission to share your code or data with third parties

Hands-on session with Copilot

GitHub Copilot

What is it?

- One of the first (maybe the first) serious AI assistants for programming
- Only vaguely related to the Windows / MS Office Copilot
  - Microsoft owns GitHub and rolled out the “Copilot” branding across its products
- Can use LLMs from many vendors (OpenAI, Google Gemini, Anthropic’s Claude)
  - Interface works the same regardless of which LLM backend is used
- GitHub offers various Copilot plans that differ in terms of limits (how many completions/queries/etc. you can run per month)
  - Limits vary across vendors/models (unlimited for “cheaper” models such as GPT-4.1)

Installation

How to set it up

- Verify that `GitHub Copilot Chat` extension is installed
- Make sure the setting Chat: Disable AI Features is unchecked
  - Search for this in Preferences/Settings Ctrl + , — Mac: Cmd + ,

Logging into your GitHub account

- You need to be signed into your GitHub account to use Copilot
- If you signed in and your model selection is very limited even though you have GitHub Pro:
  - Close and restart VS Code
  - Sign out and back in to GitHub
    - Accounts: Manage Accounts > Select your GitHub account > Sign out

Types of interactions with Copilot

Autocompletion Tab or Ctrl + > — Mac: Cmd + >

- Generates one- or multi-line suggestions in a single file
- Unlike traditional autocompletion, not limited to functions or variables that actually exist

Copilot Inline Chat Ctrl + i — Mac: Cmd + i

- Invoked directly in the editor window

Copilot Chat Window Ctrl + Alt + i — Mac: Ctrl + Cmd + i

- Modes of operation:
  1. **Ask:** Quick questions about code or concepts; get explanations and fixes; ask for help with `/help`
  2. **Plan:** Interactively plan a project without running anything
  3. **Edit:** Apply to selected files (this mode is being removed from VS Code)
  4. **Agent:** Complete high-level tasks & run automated workflows
     - Might trigger multiple premium requests
- Add context (files, selections, etc.) with #

How to provide context

- Prompt only (inline or in chat window): *"Create a function to evaluate the CRRA utility function"*
- Define the function signature and provide a docstring, let the LLM complete the rest
- Modify existing code: Prompt + code provide the context
- Write instructions in a Markdown file
  - Markdown is easy to write for humans and easy to parse for LLMs
  - This is particularly useful for long instructions that need to be structured
  - Use AI to generate the instructions themselves
- Plan mode:
  - Interactive chat to define requirements
  - Once you are satisfied with the plan, let the LLM execute it
  - Can be used to save costs:
    - Let the complex / high-reasoning (expensive) model do the planning
    - Let the simple (cheaper) model execute the plan
- Always start a new chat or clear the context for a new (unrelated) task

Other interactions

Code editor

The editor context menu contains smart actions for common tasks, e.g.:

- Explain Generates a plain-language description of the selected code
- Generate code > Fix Suggests corrections to resolve errors or issues
- Generate code > Review Gives review feedback and improvement suggestions

Terminal

- Use Ctrl + i in the terminal to get help with commands and run them

Source control tab

- Use the AI symbol to generate commit messages

Advanced use

Project-specific instructions

Use the file `.github▸copilot-instructions.md` to provide additional context:

- Technologies or libraries (e.g., pandas) that should be used
- List files/folders/etc. to be ignored

Instructions should be provided in Markdown format

GitHub Copilot CLI

- See Canvas for installation instructions
- Once installed, can be started by typing `copilot` in the terminal (e.g., the terminal integrated in VS Code)
- Requires logging into your GitHub account.
  - Run `copilot login` in the terminal; or
  - Type `/login` inside a Copilot CLI session
- You should start Copilot CLI inside your project folder, not in your user directory.
- Copilot CLI by default starts in *Agent mode*, but can be switched to *Plan mode* with Shift + Tab
- Add context using the @ symbol
- Useful commands inside the Copilot CLI:
  - `/model`: Select a different LLM
  - `/clear`: Clear the context

Best practices

- Use LLM coding assistants with git version control: always commit before applying larger changes, as the LLM might wreck your entire code base.
- Always clear the context for new tasks to avoid context rot.
- Use Markdown files for longer instructions.
- Be careful with Jupyter notebooks; GitHub Copilot tends to ruin those (at least clear all output cells before feeding them into an LLM).
- Don't waste your premium quota on Claude Opus; cheaper models will do (Claude Sonnet 4.6, Gemini 3 Pro, GPT 5.3 Codex).

Useful tutorials

VS Code Copilot keeps changing fast, so some of this content may already be outdated.

- *GitHub Copilot 101: essential features* [YouTube Link]
  Demonstrates Copilot basics using a simple Python application.
- *Ask, Edit, & Agent - In-depth Overview of GitHub Copilot Chat Modes* [YouTube Link]
  Detailed demo of Copilot Chat modes (using a web application written in TypeScript)